Lab Report: Week 2 - Spring Constant

Chhaya Ramnani (UID: u8336188)

Lab partners: Tsz Wing Candice Ng, Chinatsu Komada

2026-08-05

Imports and Functions

All calculations were done in Python. Starting with imports:

```
import matplotlib.pyplot as plt
from uncertainties import ufloat
import numpy as np
from scipy import odr
import pandas as pd
```

Defining the gravity constant in Canberra:

```
g = 9.796
```

To handle our data properly, we used a custom Python function (`odr_fit`) leveraging `scipy.odr`. Because our measurements have uncertainties in both the x and y axes, standard linear regression won't cut it. This function extracts the nominal values and errors from our `uncertainties` objects, sets up a linear model ($y = mx + c$), and performs an ODR to find the best-fit slope, its uncertainty, and the y-intercept.

```
def odr_fit(x_data, y_data):
    x = np.array([v.n for v in x_data])
    sx = np.array([v.s for v in x_data])
    y = np.array([v.n for v in y_data])
    sy = np.array([v.s for v in y_data])

    linear_model = odr.Model(lambda p, x: p[0] * x + p[1])
    data = odr.RealData(x, y, sx=sx, sy=sy)

    odr_result = odr.ODR(data, linear_model, beta0=[10.0, 0.0]).run()

    slope = odr_result.beta[0]
```

```
    slope_err = odr_result.sd_beta[0]
    intercept = odr_result.beta[1]
    intercept_err = odr_result.sd_beta[1]

    return slope, slope_err, intercept, intercept_err, (x, y, sx, sy)
```

To visualize our results, we wrote a plotting function (`plot`) using `matplotlib`. This function creates a clear scatter plot of our data complete with error bars on both axes, overlays the best-fit line calculated from our ODR regression, and formats the legend to display our final spring constant value and its uncertainty.

```
def plot(
    x,
    y,
    sx,
    sy,
    slope,
    slope_err,
    intercept,
    xlabel,
    ylabel,
    title,
    filename,
):
    x_fit = np.linspace(min(x), max(x), 100)
    y_fit = slope * x_fit + intercept

    plt.figure(figsize=(7, 5))
    plt.errorbar(
        x,
        y,
        xerr=sx,
        yerr=sy,
        fmt="o",
        color="navy",
        ecolor="crimson",
        capsize=3,
        label="Experimental Data",
    )
    plt.plot(
        x_fit,
        y_fit,
        "k--",
```

```
        label=f"Fit: $k = {slope:.2f} \\pm {slope_err:.2f}$ N/m",
    )
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.title(title)
    plt.grid(True, alpha=0.6)
    plt.legend()
    plt.tight_layout()
    plt.savefig(filename, dpi=300)
    plt.show()
```

To check if both methods gave consistent results, we run a sigma test (`sigma_test`). It calculates the statistical difference between the two spring constant values relative to their combined uncertainties. If the difference is less than or equal to 3 standard deviations (3σ), it indicates our results agree within experimental error.

```
def sigma_test(k1, sk1, k2, sk2):
    sigma_diff = abs(k1 - k2) / np.sqrt(sk1**2 + sk2**2)
    print("SIGMA TEST")
    print(f"Static k  : {k1:.3f} +/- {sk1:.3f} N/m")
    print(f"Dynamic k : {k2:.3f} +/- {sk2:.3f} N/m")
    print(f"Discrepancy: {sigma_diff:.2f} sigma")

    if sigma_diff <= 3.0:
        print("results AGREE within experimental uncertainty")
    else:
        print("results DISAGREE")
```

EXPERIMENT 1

Aim

To find the spring constant of the spring provided using the static method: measuring the extension for a given applied force.

Method

We start with Hooke's Law:

$$F = kx$$

Where $F = \text{Force}$, $k = \text{spring constant}$, and $x = \text{displacement}$.

For an initial length x_0 and the displacement x, the extension of the string is given by $(x_0 - x)$.

Equating the two equations for F:

$$mg = k(x_0 - x)$$

Comparing this to the linear equation $y = mx + c$, we get:

$$y = mg$$

$$x_{\text{axis}} = (x_0 - x)$$

Therefore, the spring constant k corresponds to the gradient of the slope.

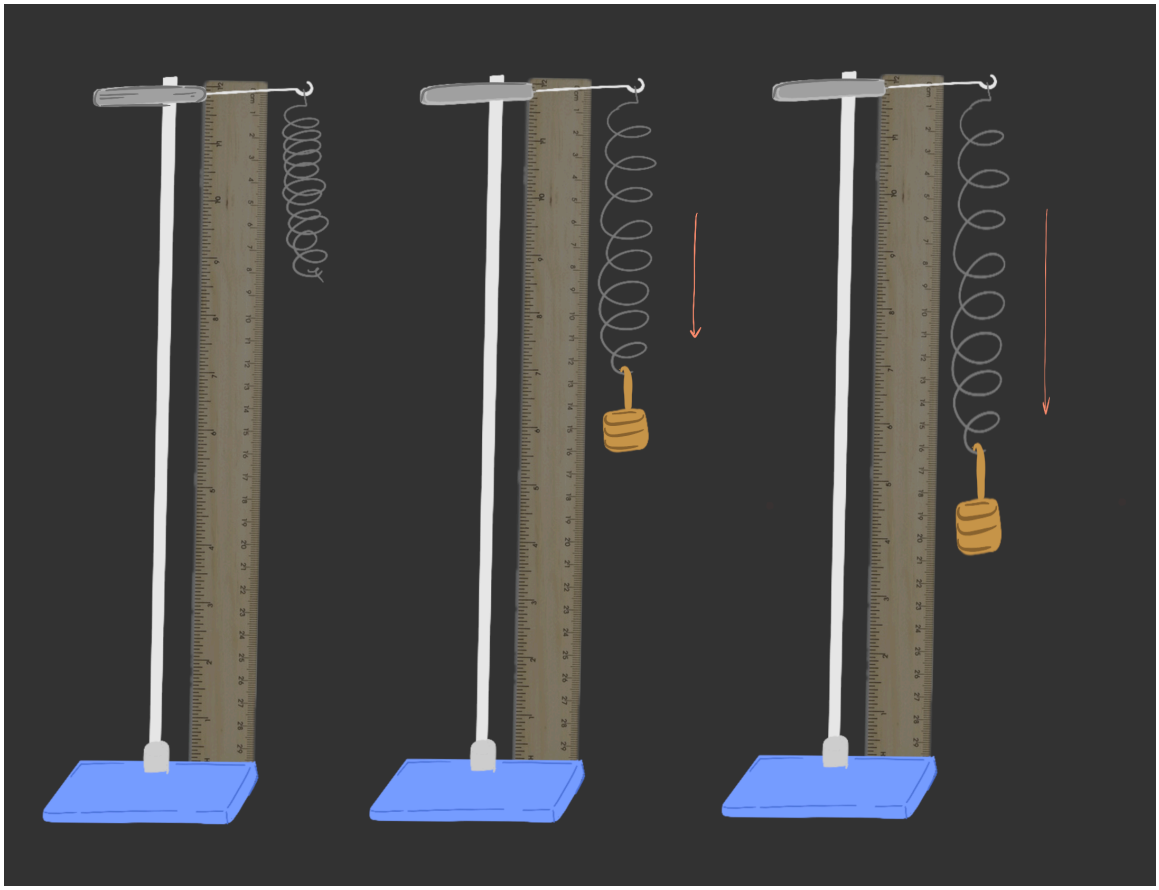


Figure 1: static setup

Procedure

We start by measuring the unstretched length x_0 of the spring that we will use to dangle the weights from. Since we are using a ruler to measure the length in half centimeters, we take 0.25 cm as the uncertainty in measurement.

```
x0 = ufloat(0.30, 0.0025)       # initial spring length (m)
print(f"x0 = {x0} m")
```

```
x0 = 0.3000+/-0.0025 m
```

We then measured the mass of our initial mass slot and consequently each mass slot that was added = $50g$. Accounting for the fluctuation of exactly 1 gram,

```
m_slot = ufloat(0.050, 0.0001)  # mass per slot (kg)
print(f"m_slot = {m_slot} kg")
```

```
m_slot = 0.05000+/-0.00010 kg
```

```
total_lengths_cm = [32.4,
                    34.7,
                    37.1,
                    39.4,
                    41.8,
                    44.3,
                    46.5,
                    48.9,
                    51.5,
                    53.8]
```

Because our measurements are in cm, we convert them to m and also input the uncertainties

```
total_lengths_m = [ufloat(L / 100.0, 0.0025) for L in total_lengths_cm]
```

Now, calculating force and extension:

```
total_masses = [i * m_slot for i in range(1, 11)]
            # computes cumulative mass for each step
force = [m * g for m in total_masses]
            # multiplies total mass by 'g' to get downward force
extension = [L - x0 for L in total_lengths_m]
            # measured lengths after mass suspended - initial length (x_0)
```

	Slots	Mass (kg)	Length (m)	Extension (m)	Force (N)
0	1	0.0500 ± 0.0001	0.3240 ± 0.0025	0.0240 ± 0.0035	0.4898 ± 0.0010
1	2	0.1000 ± 0.0002	0.3470 ± 0.0025	0.0470 ± 0.0035	0.9796 ± 0.0020
2	3	0.1500 ± 0.0003	0.3710 ± 0.0025	0.0710 ± 0.0035	1.4694 ± 0.0029
3	4	0.2000 ± 0.0004	0.3940 ± 0.0025	0.0940 ± 0.0035	1.9592 ± 0.0039
4	5	0.2500 ± 0.0005	0.4180 ± 0.0025	0.1180 ± 0.0035	2.4490 ± 0.0049
5	6	0.3000 ± 0.0006	0.4430 ± 0.0025	0.1430 ± 0.0035	2.9388 ± 0.0059

	Slots	Mass (kg)	Length (m)	Extension (m)	Force (N)
6	7	0.3500 ± 0.0007	0.4650 ± 0.0025	0.1650 ± 0.0035	3.4286 ± 0.0069
7	8	0.4000 ± 0.0008	0.4890 ± 0.0025	0.1890 ± 0.0035	3.9184 ± 0.0078
8	9	0.4500 ± 0.0009	0.5150 ± 0.0025	0.2150 ± 0.0035	4.4082 ± 0.0088
9	10	0.5000 ± 0.0010	0.5380 ± 0.0025	0.2380 ± 0.0035	4.8980 ± 0.0098

Using the plot function defined in **Imports and Functions** and combining everything we have,

```
def static_method():
    x0 = ufloat(0.30, 0.0025)       # initial spring length (m)
    m_slot = ufloat(0.050, 0.0001)  # mass per slot (kg)

    # measured total lengths (cm)
    total_lengths_cm = [32.4,
                        34.7,
                        37.1,
                        39.4,
                        41.8,
                        44.3,
                        46.5,
                        48.9,
                        51.5,]
    total_lengths_m = [ufloat(L / 100.0, 0.0025) for L in total_lengths_cm]
    total_masses = [i * m_slot for i in range(1, 10)]
    force = [m * g for m in total_masses]
    extension = [L - x0 for L in total_lengths_m]

    k, sk, intercept, intercept_err, plot_data = odr_fit(extension, force)
    x, y, sx, sy = plot_data
    print("static method xerrs and yerrs")
    for i in range(len(x)):
          print(f"point {i+1}: x = {x[i]:.4f} ± {sx[i]:.4f} | y = {y[i]:.4f}
± {sy[i]:.4f}")
    print(f"slope (k)     = {k:.4f} ± {sk:.4f} N/m")
    print(f"intercept     = {intercept:.4f} ± {intercept_err:.4f} N")

    plot(
        *plot_data,
        k,
        sk,
        intercept,
        xlabel="Extension $x$ (m)",
```

```
        ylabel="Applied Force $mg$ (N)",
        title="Static Determination of Spring Constant ($k$)",
        filename="/Users/chhaya/Documents/uniwork-cr/year 1/sem 2/phys1201/week
2/graphs/static_k_plot.png",
    )

    return k, sk, intercept, intercept_err, total_masses
```

EXPERIMENT 2

Aim

To find the spring constant of the spring provided using the dynamic method: measuring the period for different applied forces

Method

Applying Hooke's Law, we have been given the following equation:

$$T = 2\pi\sqrt{\frac{m + \frac{m_s}{3}}{k}}$$

where T is the period of the bounce, m is the mass, m_s is the mass of the spring and k is the spring constant.

Rearranging, we get

$$k\left(\frac{T}{2\pi}\right)^2 = m + \frac{m_s}{3}$$

Comparing to $y = mx + c$, we get:

$$y = m + \frac{m_s}{3}$$

$$x = \left(\frac{T}{2\pi}\right)^2$$

And the gradient will be k.

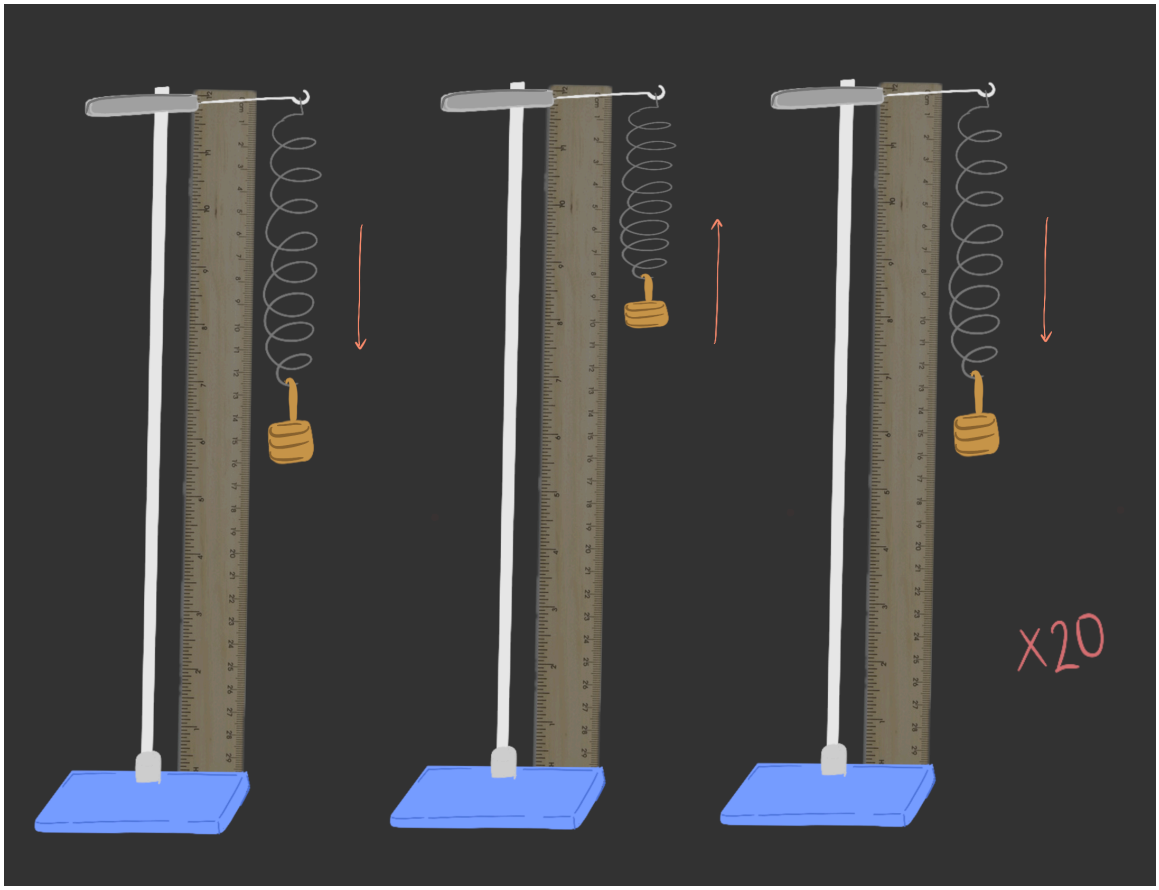


Figure 2: dynamic setup

Procedure

We have everything we need for this recipe, except T. We can obtain it by measuring the time taken by the spring to bounce 20 times. We choose to count bounces rather than time because we can get a more accurate measurement if we only have to keep track of the number of bounces, rather than tracking both, say, 20 bounces (19 periods) and 20 seconds.
The ground for errors is further minimized by having one person starting the timer and initiating the bounce.

We first started by measuring the mas of the spring on the laboratory scale. We account for the slight seldom fluctuation of the mass as the uncertainity.

```
spring_mass = ufloat(0.0473, 0.000001)  # spring mass in kg
m_slot = ufloat(0.050, 0.0001)  # mass per slot (kg)
total_masses = [i * m_slot for i in range(1, 10)] # mass per addition of weight
print(f"spring_mass = {spring_mass} kg")
print(f"m_slot = {m_slot} kg")
```

```
spring_mass = 0.0473000+/-0.0000010 kg
m_slot = 0.05000+/-0.00010 kg
```

Then, we measure the time taken in seconds for each mass to bounce 20 times and we get

```
time_20_bounces_s =   [7.55,
                       9.05,
                       10.85,
                       12.48,
                       13.75,
                       15.38,
                       16.53,
                       17.64,
                       18.28]
```

To find period, we divide time recorded for 20 bounces by 19 to find the period. Taking 0.2s as an uncertainty accounting for delay in reflex, since the reflex delay applies once at the start and once at the stop of the stopwatch.

```
periods = [ufloat(t, 0.20) / 19.0 for t in time_20_bounces_s]
print(periods)
```

```
[0.3973684210526316+/-0.010526315789473684,
0.47631578947368425+/-0.010526315789473684,
0.5710526315789474+/-0.010526315789473684,
0.6568421052631579+/-0.010526315789473684,
0.7236842105263158+/-0.010526315789473684,
0.8094736842105263+/-0.010526315789473684,
0.8700000000000001+/-0.010526315789473684,
0.9284210526315789+/-0.010526315789473684,
0.9621052631578948+/-0.010526315789473684]
```

From the **Method** section,
we had $x = (\frac{T}{2\pi})^2$, so scaling our values from the `periods` to find `x_dynamic`

```
x_dynamic = [(T / (2 * np.pi)) ** 2 for T in periods]
print(x_dynamic)
```

```
[0.003999695824496093+/-0.00021190441454283938,
0.005746854739104274+/-0.0002540046293526751,
0.00826023756325146+/-0.00030452488712447785,
0.010928542161196375+/-0.0003502737872178325,
```

```
0.013265953104140919+/-0.00038591863575682675,
0.01659761675343948+/-0.0004316675358501815,
0.01917250097472137+/-0.0004639443672043885,
0.02183384500381772+/-0.0004950985261636671,
0.02344690070085164+/-0.0005130612844825304]
```

and $y = m + \frac{m_s}{3}$, scaling from `total masses` to find `y_dynamic`

```
y_dynamic = [m + (spring_mass / 3.0) for m in total_masses]
print(y_dynamic)
```

```
[0.06576666666666667+/-0.00010000055555401236,
0.11576666666666667+/-0.00020000027777758488,
0.1657666666666667+/-0.00030000018518512806,
0.2157666666666667+/-0.0004000001388888648,
0.26576666666666665+/-0.0005000001111110987,
0.3157666666666667+/-0.0006000000925925855,
0.3657666666666667+/-0.0007000000793650748,
0.4157666666666667+/-0.0008000000694444414,
0.46576666666666666+/-0.0009000000617283931]
```

	Slots	Mass (kg)	Time for 20 bounces (s)	Period (s)	Effective mass (kg)
0	1	0.0500 ± 0.0001	7.55	0.3974 ± 0.0105	0.0658 ± 0.0001
1	2	0.1000 ± 0.0002	9.05	0.4763 ± 0.0105	0.1158 ± 0.0002
2	3	0.1500 ± 0.0003	10.85	0.5711 ± 0.0105	0.1658 ± 0.0003
3	4	0.2000 ± 0.0004	12.48	0.6568 ± 0.0105	0.2158 ± 0.0004
4	5	0.2500 ± 0.0005	13.75	0.7237 ± 0.0105	0.2658 ± 0.0005
5	6	0.3000 ± 0.0006	15.38	0.8095 ± 0.0105	0.3158 ± 0.0006
6	7	0.3500 ± 0.0007	16.53	0.8700 ± 0.0105	0.3658 ± 0.0007
7	8	0.4000 ± 0.0008	17.64	0.9284 ± 0.0105	0.4158 ± 0.0008
8	9	0.4500 ± 0.0009	18.28	0.9621 ± 0.0105	0.4658 ± 0.0009

Using the plot function defined in **Imports and Functions** and combining everything we have,

```
def dynamic_method():
    spring_mass = ufloat(0.0473, 0.000001)  # spring mass in kg
    m_slot = ufloat(0.050, 0.0001)  # mass per slot (kg)
    total_masses = [i * m_slot for i in range(1, 10)]
```

```
    time_20_bounces_s = [7.55,
                         9.05,
                         10.85,
                         12.48,
                         13.75,
                         15.38,
                         16.53,
                         17.64,
                         18.28]
    periods = [ufloat(t, 0.20) / 20.0 for t in time_20_bounces_s]
    print(periods)
    x_dynamic = [(T / (2 * np.pi)) ** 2 for T in periods]
    y_dynamic = [m + (spring_mass / 3.0) for m in total_masses]

    k, sk, intercept, intercept_err, plot_data = odr_fit(x_dynamic, y_dynamic)
    x, y, sx, sy = plot_data

    print("dynamic method xerrs and yerrs")
    for i in range(len(x)):
          print(f"point {i+1}: x = {x[i]:.4f} ± {sx[i]:.4f} | y = {y[i]:.4f}
± {sy[i]:.4f}")
    print(f"slope (k)     = {k:.4f} ± {sk:.4f} N/m")
    print(f"intercept     = {intercept:.4f} ± {intercept_err:.4f} kg")

    plot(
      *plot_data,
        k,
        sk,
        intercept,
        xlabel=r"$(T / 2\pi)^2$ ($\text{s}^2$)",
        ylabel=r"Effective Mass $m + m_s/3$ (kg)",
        title="Dynamic Determination of Spring Constant ($k$)",
        filename="/Users/chhaya/Documents/uniwork-cr/year 1/sem 2/phys1201/week
2/graphs/dynamic_k_plot.png",
    )

    return k, sk, intercept, intercept_err
```

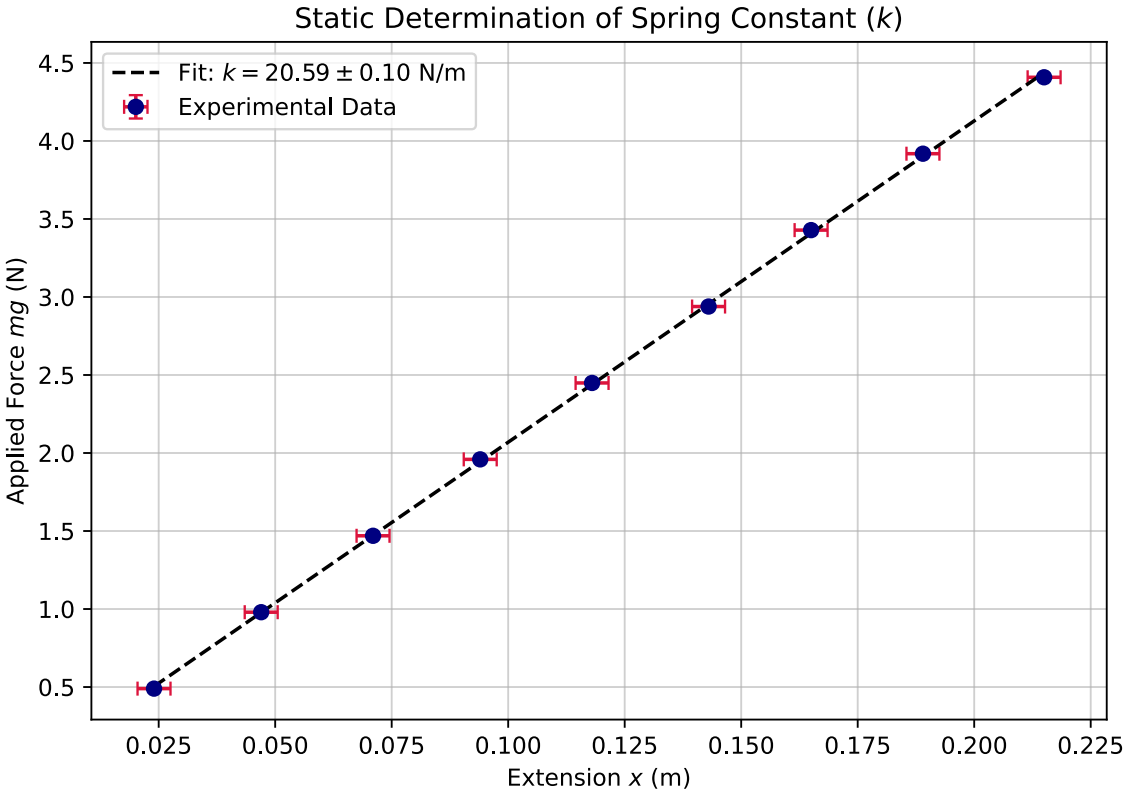
Results and Observations

```
k_static, sk_static, intercept_static, intercept_err_static, total_masses =
static_method()
print(f"static method result: k = {k_static:.3f} +/- {sk_static:.3f} N/m")
```

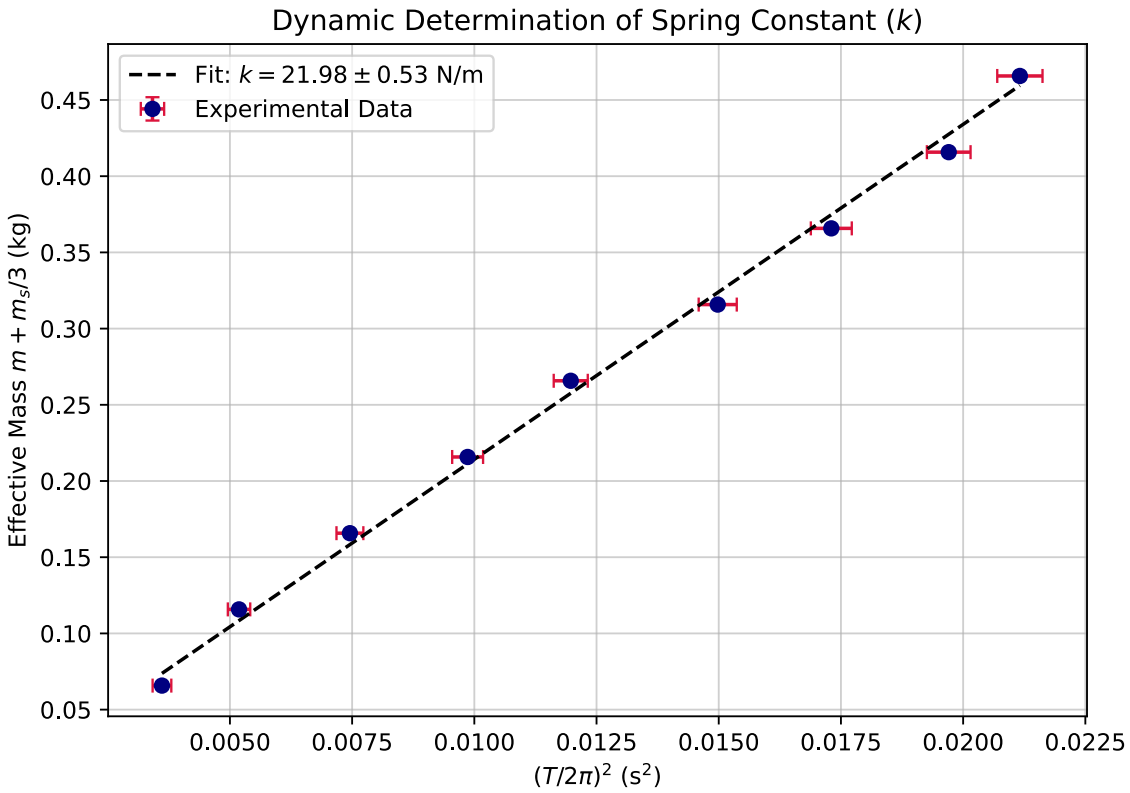
```
k_dynamic,    sk_dynamic,    intercept_dynamic,    intercept_err_dynamic    =
dynamic_method()
print(f"dynamic method result: k = {k_dynamic:.3f} +/- {sk_dynamic:.3f} N/m")

sigma_test(k_static, sk_static, k_dynamic, sk_dynamic)
```

```
static method xerrs and yerrs
point 1: x = 0.0240 ± 0.0035 | y = 0.4898 ± 0.0010
point 2: x = 0.0470 ± 0.0035 | y = 0.9796 ± 0.0020
point 3: x = 0.0710 ± 0.0035 | y = 1.4694 ± 0.0029
point 4: x = 0.0940 ± 0.0035 | y = 1.9592 ± 0.0039
point 5: x = 0.1180 ± 0.0035 | y = 2.4490 ± 0.0049
point 6: x = 0.1430 ± 0.0035 | y = 2.9388 ± 0.0059
point 7: x = 0.1650 ± 0.0035 | y = 3.4286 ± 0.0069
point 8: x = 0.1890 ± 0.0035 | y = 3.9184 ± 0.0078
point 9: x = 0.2150 ± 0.0035 | y = 4.4082 ± 0.0088
slope (k)     = 20.5948 ± 0.0981 N/m
intercept     = 0.0097 ± 0.0131 N
```



```
static method result: k = 20.595 +/- 0.098 N/m
[0.3775+/-0.010000000000000002,                 0.4525+/-0.010000000000000002,
0.5425+/-0.010000000000000002,                  0.624+/-0.010000000000000002,
0.6875+/-0.010000000000000002,                  0.769+/-0.010000000000000002,
0.8265+/-0.010000000000000002,
0.882+/-0.010000000000000002, 0.914+/-0.010000000000000002]
dynamic method xerrs and yerrs
point 1: x = 0.0036 ± 0.0002 | y = 0.0658 ± 0.0001
point 2: x = 0.0052 ± 0.0002 | y = 0.1158 ± 0.0002
point 3: x = 0.0075 ± 0.0003 | y = 0.1658 ± 0.0003
point 4: x = 0.0099 ± 0.0003 | y = 0.2158 ± 0.0004
point 5: x = 0.0120 ± 0.0003 | y = 0.2658 ± 0.0005
point 6: x = 0.0150 ± 0.0004 | y = 0.3158 ± 0.0006
point 7: x = 0.0173 ± 0.0004 | y = 0.3658 ± 0.0007
point 8: x = 0.0197 ± 0.0004 | y = 0.4158 ± 0.0008
point 9: x = 0.0212 ± 0.0005 | y = 0.4658 ± 0.0009
slope (k)     = 21.9785 ± 0.5305 N/m
intercept     = -0.0056 ± 0.0056 kg
```



```
dynamic method result: k = 21.979 +/- 0.530 N/m
SIGMA TEST
Static k  : 20.595 +/- 0.098 N/m
Dynamic k : 21.979 +/- 0.530 N/m
Discrepancy: 2.56 sigma
results AGREE within experimental uncertainty
```

	Method	k (N/m)	Intercept
0	Static	20.5948 ± 0.0981	0.0097 ± 0.0131
1	Dynamic	21.9785 ± 0.5305	−0.0056 ± 0.0056

The static method gave $k_{\text{static}} = 20.60 \pm 0.10$ N/m, and the dynamic method gave $k_{\text{dynamic}} = 21.98 \pm 0.53$ N/m, a discrepancy of 2.56σ. By our 3σ threshold, the two methods **agree** within uncertainty, though the gap is still notable and the dynamic method's much larger uncertainty (roughly 5x that of the static method) reflects that timing measurements are inherently less precise than the ruler-based length measurements.

Likely contributors to this discrepancy:

- **Reflex-timing uncertainty.** Timing a trial means starting the stopwatch on the first bounce and stopping it on the 20th, so there are two separate moments where reaction time can throw off the reading, not just one. A typical human reaction time is around $+/-0.1s$, so with two independent chances for that delay to creep in, the total timing uncertainty is closer to $+/-0.2s$ than the $+/-0.1s$ you'd get from only counting one reaction. This matters because $(T/2\pi)^2$ is the x-value in the dynamic fit — if this uncertainty is underestimated, the fit's error bars end up too tight, making the two methods look more disagreeing than they really are.
- **The $m_s/3$ effective mass correction is itself an approximation**, exact only for an idealised uniform spring. Real lab springs are not perfectly uniform, and this could introduce a small systematic offset between the two methods.
- **Air resistance causing the spring to sway slightly while bouncing, and errors introduced by human obersevation (eye-level reading and counting vertical oscillations)**